



Figure 1: ACID (Credits: <https://morpheusdata.com/cloud-blog/when-do-you-need-acid-compliance/>)

- ABC credit ₹100
- Transaction commit

In the above example, we have a ₹100 transaction taking place. If anything fails during the process, the ₹100 will be debited from XYZ but not credited to ABC. Obviously, this is not an ideal situation.

There is another type of transaction which is read only but is still required to be termed as a transaction. We call it a read-only transaction. Let's explore its requirement through an example.

- Without transaction:
 - Count of number of deals booked -> Job 1
 - 1 deal got cancelled -> Job 2
 - Count of dealers along with count of deals booked -> Continuation of Job 1
 - Here you see there will be a conflict in the answers produced by Job 1
- With transaction
 - Transaction begin -> Job 1
 - Count of number of deals booked -> Job 1
 - Count of dealers along with deal booked -> Continuation of Job 1
 - Transaction commit

In this case, all the reads within this transaction get consistent data from the database, even if interrupted by other jobs. The current transaction remains unaffected. This is particularly useful while building a huge report or taking a snapshot.

Atomicity

Atomicity represents a single unit — it should not break. All the queries within a transaction must succeed; otherwise nothing occurs, and all changes should be rolled back. There are two types of issues that can cause failure during a transaction.

- Query failure, such as duplicate primary key insert or updating invalid data
- Database failure, such as a hardware crash or power issues

In the case of query failure, we still have the control and can perform the rollback. However, database failures such as hardware crash or crash while performing a commit are serious issues that need to be handled sensitively.

Some databases write directly to the disk and mark the transaction as successful at the end. This approach ensures commit is faster but carries the risk of bad data. To avoid the bad data, some databases copy the old data before applying changes, a technique known as read-copy-update. However, this rapidly increases hard disk space usage.

Other databases maintain changes in the RAM and write in the database only during success. Here the rollback is faster but can lead to issues such as RAM overload and RAM crash. As they say, there's no free lunch; we need to sacrifice either fast write or fast rollback in these two scenarios.

Typically, several systems implement atomicity through journaling. The system synchronises the logs as necessary after changes have successfully taken place. After crash recovery, it ignores incomplete entries. Different database implementations may vary depending on many factors such as:

- Concurrency issues, allowing the threads of users working on the same data using a lock mechanism. Having time gaps to avoid deadlocks.
- Control of the number of operations included in a single transaction. Here both the throughput and the possibility of deadlock are controlled. The more the operations included in a transaction, the more likely it is that a transaction will block other operations and a deadlock will occur.
- Starting databases only when stale transactions are clear; in some databases they start immediately, and crash changes are fixed in parallel.

However, the principle of atomicity — complete success or complete failure — remains.

Consistency

Consistency means data is in a consistent state when a transaction starts and when it ends. Any progress made from one valid state to another should take care of auto trigger changes, ensure constraints, and so on. For example, when data transfers from A to B, it should verify B exists, and the values after the transaction completes should be valid without any corruption.

Consistency in reading is equally important. This is simpler in a single database where you write and read from the same database. Having distributed databases makes it complicated. If a write is committed and we read from it, if it's the same database the result would be prompt, but in a distributed database the write needs to be reached from where it is asked to read. At times if we get stale information, we apply a concept called 'eventual consistency'. This says that while the data is not consistent, it will provide the required information after the sync. Early results of eventual